Task-7:- Documentation-Automation for Temp Directory File Count Monitoring (AWS Lambda → Pushgateway → Grafana)

1. Project Overview

This project implements an automated monitoring solution to track the number of files present inside a Windows system temporary directory and visualize the collected metrics using an observability stack.

The solution leverages AWS serverless services, containerization, and Prometheus-based monitoring to provide automated metric collection, visualization, and alerting capabilities.

The monitoring pipeline automatically collects file count metrics from a Windows EC2 instance, publishes them to Prometheus Pushgateway, visualizes data in Grafana dashboards, and generates email alerts when predefined thresholds are exceeded.

Additionally, the collector application was containerized and deployed using both AWS Lambda Container Images and ECS Fargate to demonstrate reusable deployment architecture.

---

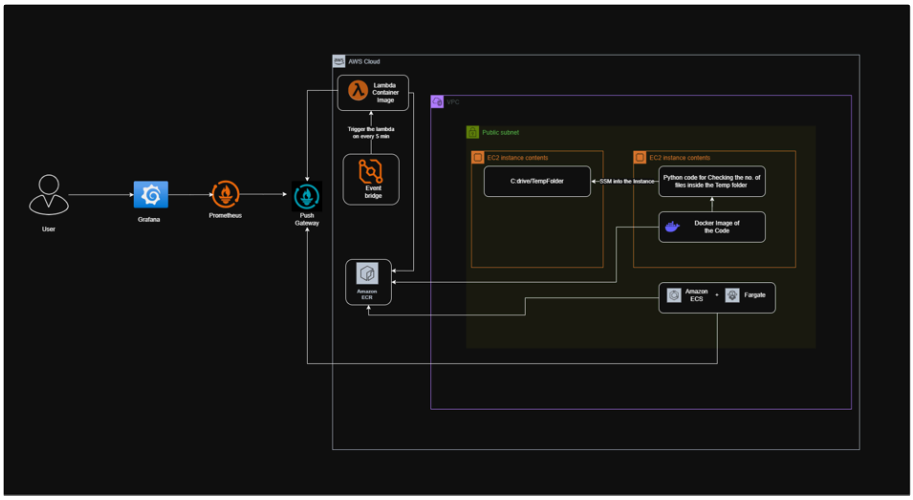
2. Objectives Achieved

The following objectives were successfully implemented:

- Automated monitoring of Windows Temp directory
- Scheduled metric collection using AWS Lambda
- Metric publishing using Prometheus Pushgateway
- Metric scraping using Prometheus
- Visualization using Grafana dashboards
- Email alerting based on threshold conditions
- Containerized collector deployment
- Dual deployment using Lambda Container and ECS Fargate

---

3. Solution Architecture



High-Level Architecture

```
Windows EC2 Instance
        |
        | AWS Systems Manager (SSM)
        ▼
AWS Lambda Collector
        ▼
Prometheus Pushgateway
        ▼
Prometheus Server
        ▼
Grafana Dashboard
        ▼
```

```
Email Alert Notification
```

Bonus Architecture Extension

```
Containerized Collector Image
          |
  ┌─────────┴─────────┐
  │                   │
Lambda Container   ECS Fargate Task
```

---

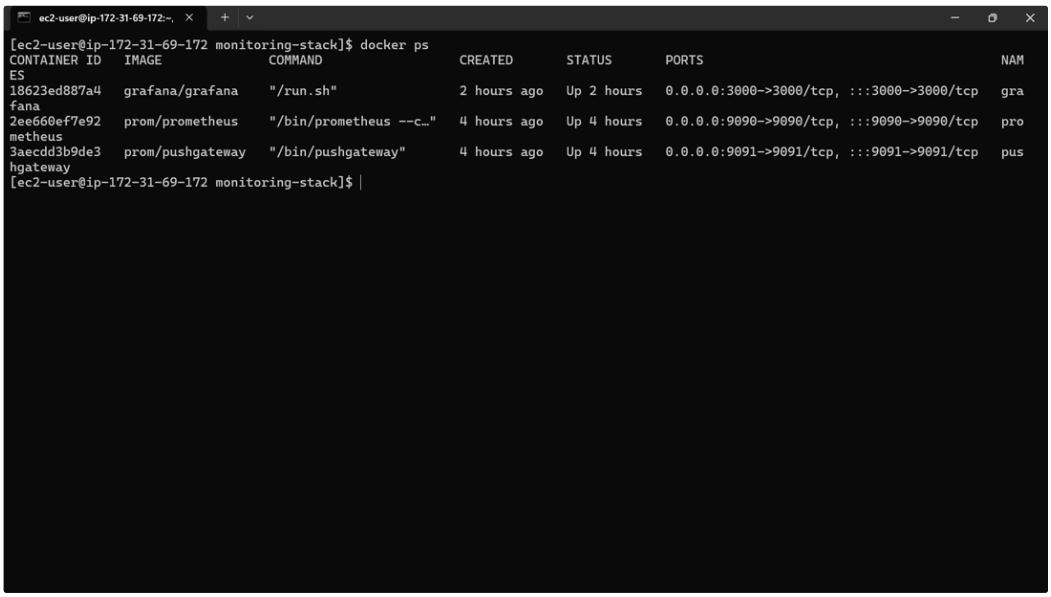
4. Infrastructure Setup

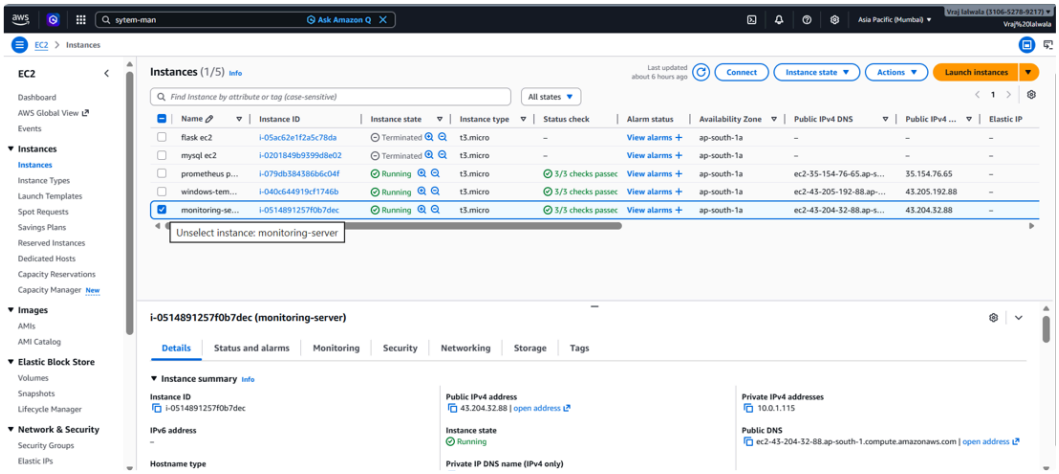
A dedicated monitoring EC2 instance was provisioned to host the observability stack.

Components Installed

Component	Purpose
Docker	Container runtime
Pushgateway	Metric ingestion
Prometheus	Metric scraping
Grafana	Visualization & alerting

All monitoring services were deployed as Docker containers to ensure portability and simplified management.





- `docker ps` output showing containers running

---

5. Collector Implementation

Collector Functionality

The monitoring collector performs the following operations:

1. Executes a PowerShell command remotely using AWS Systems Manager.
2. Reads the Windows temporary directory.
3. Counts total files present.
4. Converts output into Prometheus-compatible metric format.
5. Pushes metric data to Pushgateway.

Temp Directory Command

```
(Get-ChildItem -Path C:\Users\Administrator\AppData\Local\Temp -File | Measure-Object).Count
```

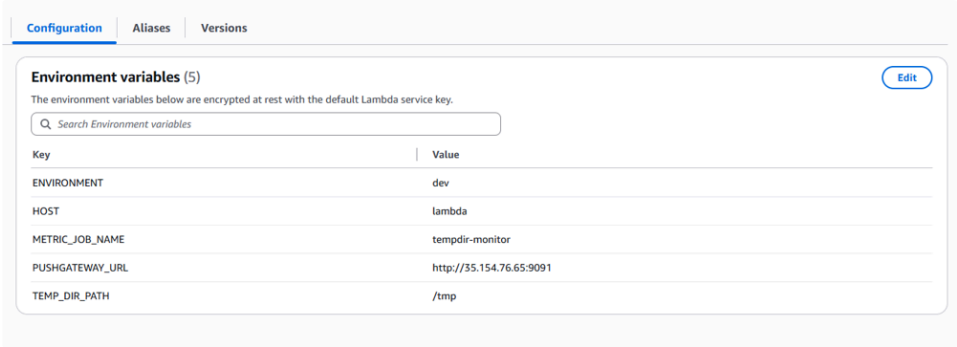
Generated Metric

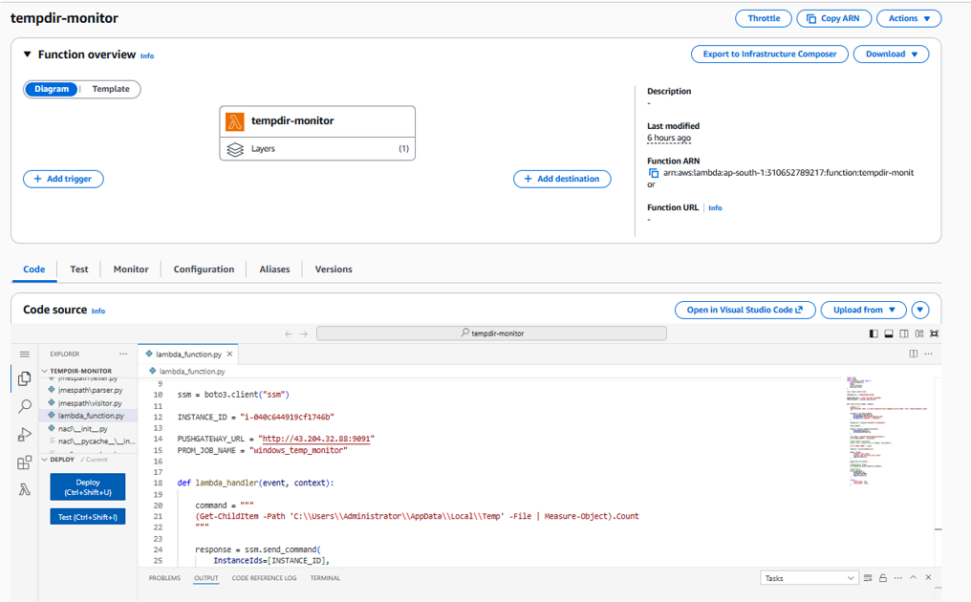
```
temp_dir_file_count
```

Configuration Externalization

The collector uses environment variables:

Variable	Description
TEMP_DIR_PATH	Target directory
PUSHGATEWAY_URL	Pushgateway endpoint
METRIC_JOB_NAME	Prometheus job name





---

6. Automation using EventBridge

Metric collection was automated using AWS EventBridge scheduling.

Execution Frequency:

```
rate(5 minutes)
```

This ensures periodic monitoring without manual intervention.

Pushgateway Integration

The collector pushes metrics using the Prometheus Python client.

Example Push Operation:

```
push_to_gateway(
    PUSHGATEWAY_URL,
    job="windows_temp_monitor",
    registry=registry
)
```

Pushgateway exposes metrics via HTTP endpoint which is later scraped by Prometheus.

```
# HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 3.1006e-05
go_gc_duration_seconds{quantile="0.25"} 0.000111944
go_gc_duration_seconds{quantile="0.5"} 0.000127253
go_gc_duration_seconds{quantile="0.75"} 0.000143789
go_gc_duration_seconds{quantile="1"} 0.001628623
go_gc_duration_seconds_sum 0.036133351
go_gc_duration_seconds_count 239
# HELP go_gc_gogc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the GOGC environment variable, and the runtime/debug.SetGCPercent function. Sourced from /gc/gogc:percent.
# TYPE go_gc_gogc_percent gauge
go_gc_gogc_percent 100
# HELP go_gc_gomemlimit_bytes Go runtime memory limit configured by the user, otherwise math.MaxInt64. This value is set by the GOMEMLIMIT environment variable, and the runtime/debug.SetMemoryLimit function. Sourced from
/gc/gomemlimit:bytes.
# TYPE go_gc_gomemlimit_bytes gauge
go_gc_gomemlimit_bytes 9.223372036854776e+18
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 12
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.25.3"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated in heap and currently in use. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 5.312344e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated in heap until now, even if released already. Equals to /gc/heap/allocs:bytes.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 3.79690024e+08
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. Equals to /memory/classes/profiling/buckets:bytes.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.482604e+06
# HELP go_memstats_frees_total Total number of heap objects frees. Equals to /gc/heap/frees:objects + /gc/heap/tiny/allocs:objects.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 2.56429e+06
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata. Equals to /memory/classes/metadata/other:bytes.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 3.348752e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and currently in use, same as go_memstats_alloc_bytes. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 5.312344e+06
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used. Equals to /memory/classes/heap/released:bytes + /memory/classes/heap/free:bytes.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 4.898816e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use. Equals to /memory/classes/heap/objects:bytes + /memory/classes/heap/unused:bytes
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 6.995968e+06
# HELP go_memstats_heap_objects Number of currently allocated objects. Equals to /gc/heap/objects:objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 4734
# HELP go_memstats_heap_released_bytes Number of heap bytes released to OS. Equals to /memory/classes/heap/released:bytes.
# TYPE go_memstats_heap_released_bytes gauge
```


---

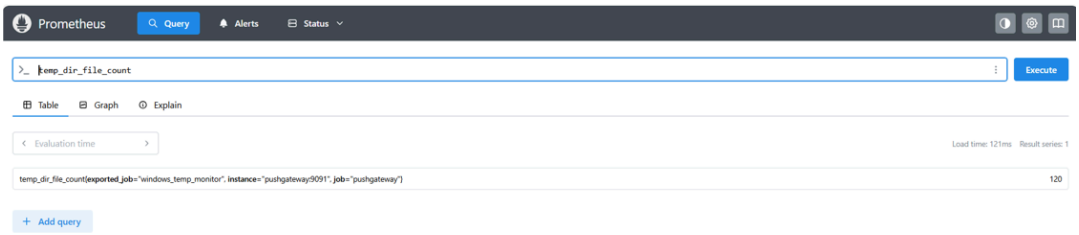
8. Prometheus Monitoring

Prometheus was configured to scrape Pushgateway periodically.

Scrape Configuration

```
scrape_configs:
  - job_name: pushgateway
    static_configs:
      - targets: ['pushgateway:9091']
```

Prometheus successfully collected time-series data from Pushgateway.



Query Used:

```
temp_dir_file_count
```

---

9. Grafana Visualization

Grafana dashboards were created for real-time visualization.

Panels Implemented

Time Series Panel

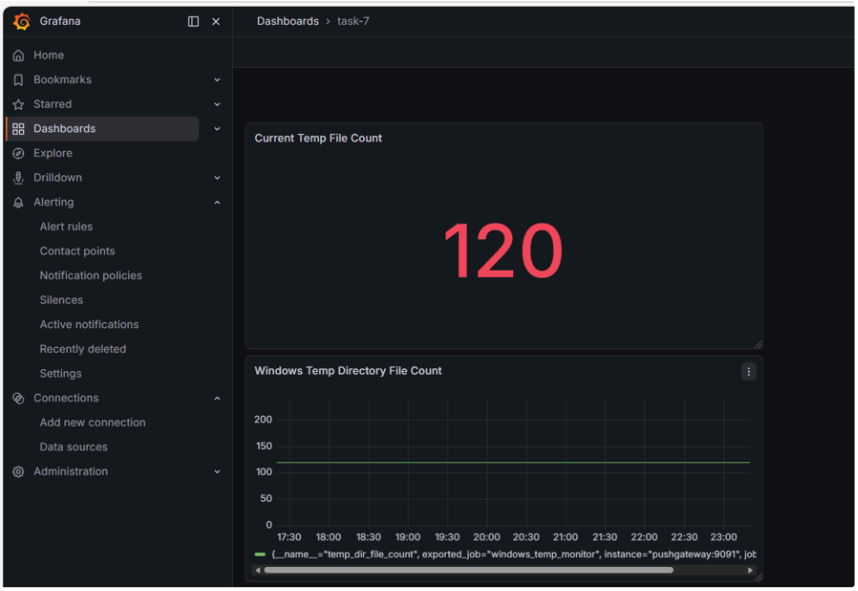
Displays historical file count trends.

Stat Panel

Displays current file count.

Threshold visualization:

- Green: Normal (<100)
- Red: Critical (>100)



10. Alerting Configuration

Grafana alerting was configured to notify via email when file count exceeds threshold.

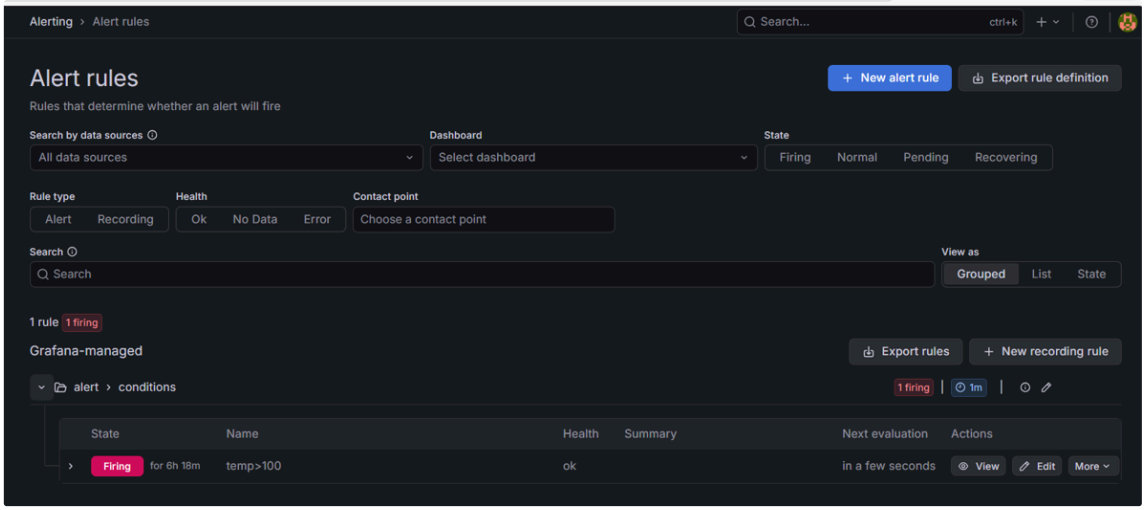
Alert Rule

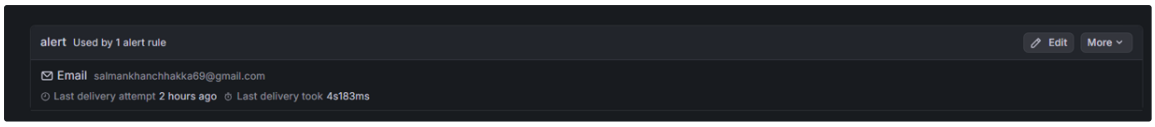
```
temp_dir_file_count > 100
```

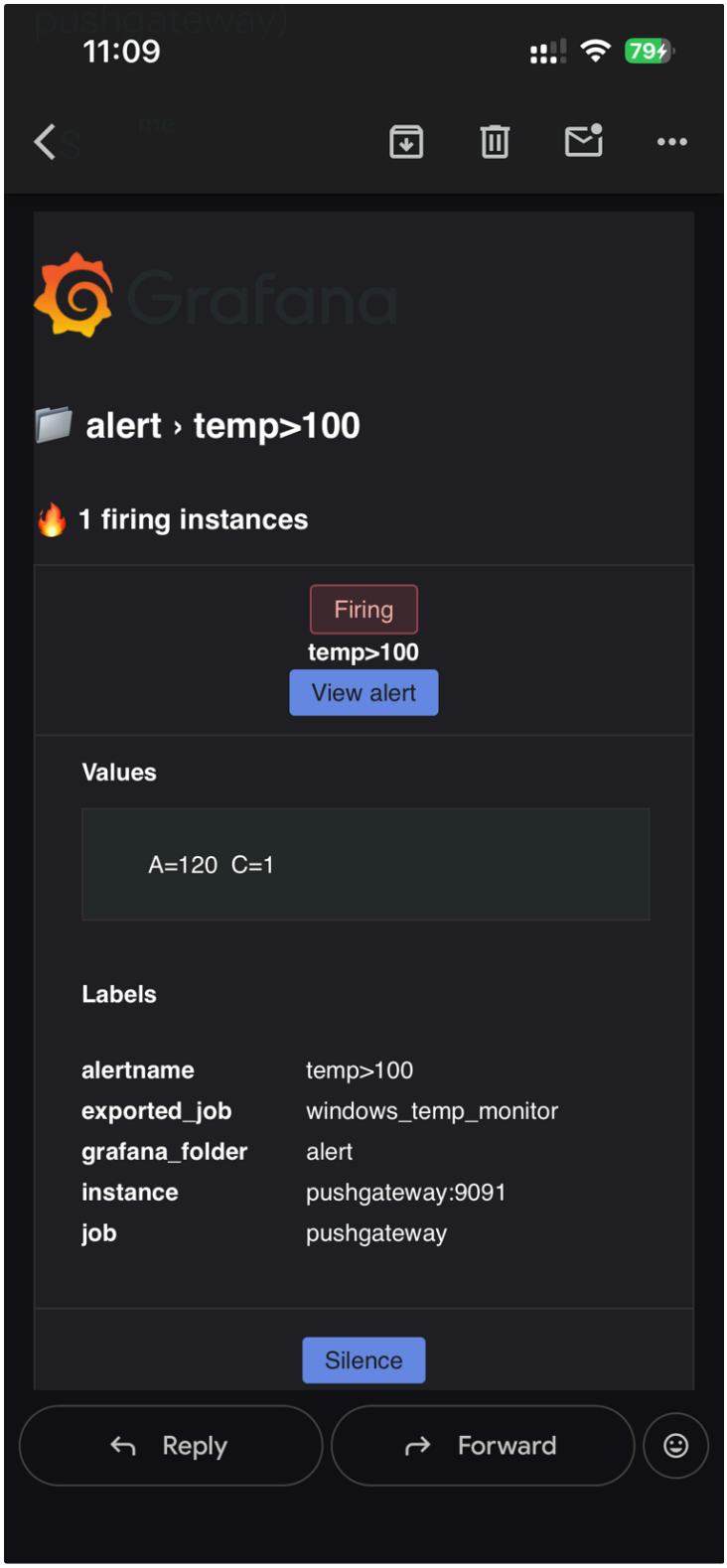
Evaluation Window:

```
5 minutes
```

Email notification configured using SMTP integration.







11. BONUS — Containerization

11.1 Collector Containerization

The monitoring collector was packaged into a Docker container.

Dockerfile

```
FROM public.ecr.aws/lambda/python:3.10
COPY collector.py .
```

```
RUN pip install boto3 prometheus_client
CMD ["collector.lambda_handler"]
```

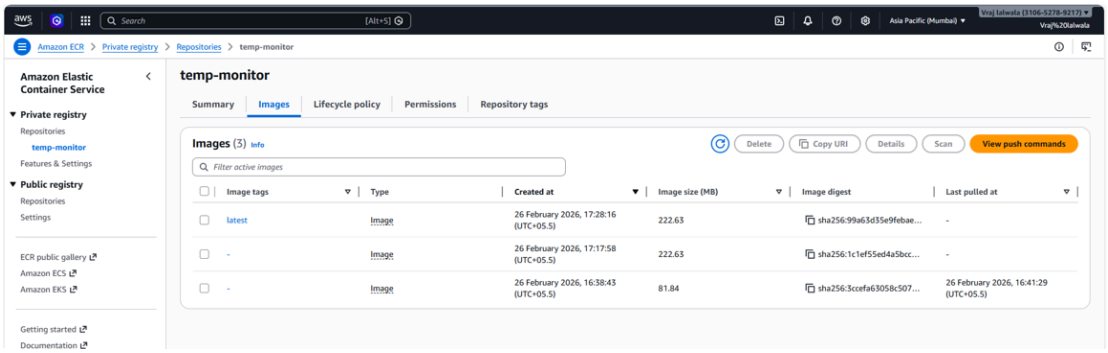
```
ubuntu@ip-10-0-1-115:~$ sudo docker build -t temp-monitor .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  303.5MB
Step 1/4 : FROM public.ecr.aws/lambda/python:3.10
3.10: Pulling from lambda/python
83e01f18b8b6: Pulling fs layer
f52d5c7658db: Pulling fs layer
e195409b2e0a: Pulling fs layer
29418cde0c9a: Pulling fs layer
f364e1d5578e: Pulling fs layer
bbca47f29bb7: Pulling fs layer
f364e1d5578e: Waiting
bbca47f29bb7: Waiting
```

11.2 Amazon ECR Integration

Container image pushed to Amazon Elastic Container Registry.

Steps performed:

- Repository creation
- Docker authentication
- Image tagging
- Image push



11.3 Lambda Container Deployment

The same container image was deployed as a Lambda function.

Benefits achieved:

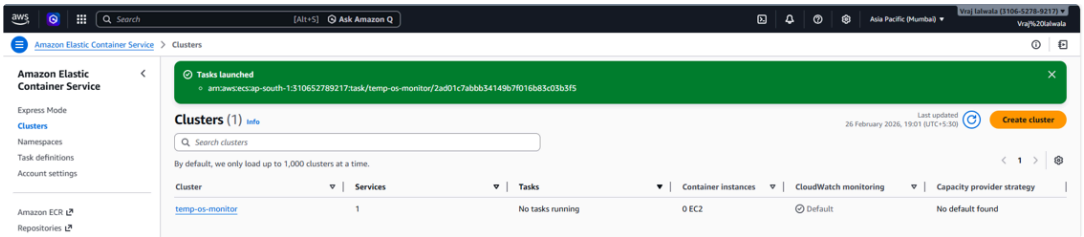
- Runtime consistency
- Dependency portability
- Simplified deployment

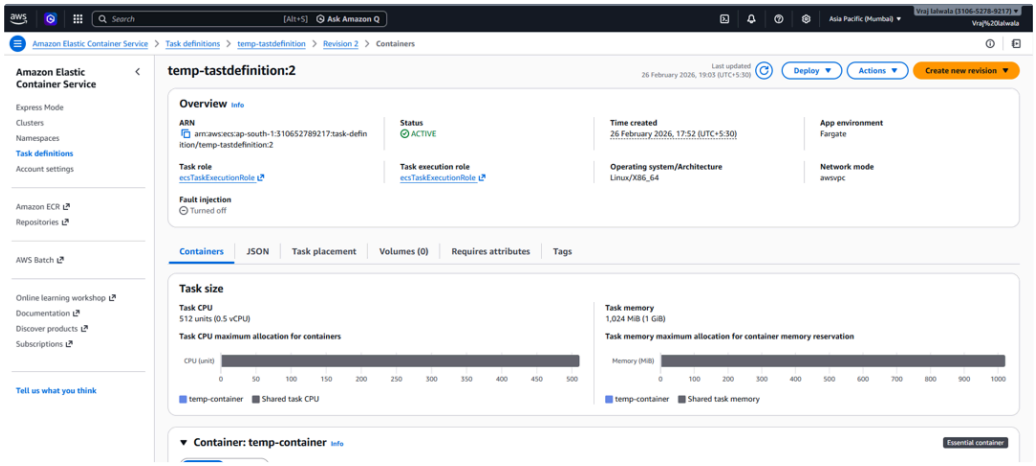
11.4 ECS Fargate Deployment

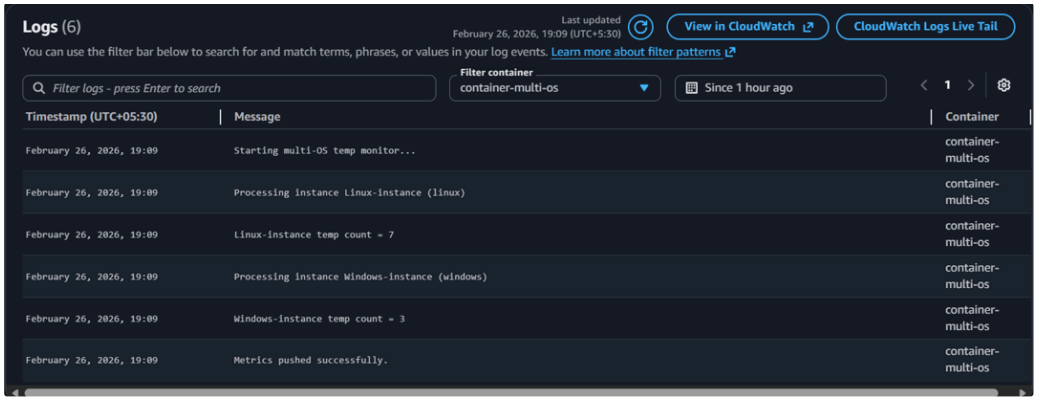
The identical container image was deployed on ECS Fargate.

Key configurations:

- ECS Cluster creation
- Task Definition setup
- IAM Task Role permissions
- CloudWatch logging enabled







11.5 Scheduled ECS Execution

ECS task execution automated using EventBridge Scheduler.

```
rate(5 minutes)
```

12. Lambda vs ECS Containerization Comparison

Feature	Lambda Container	ECS Fargate
Execution Model	Event-driven	Scheduled/Service
Scaling	Automatic	Configurable
Runtime Duration	Short-lived	Long-running
Management	Fully serverless	Container orchestration
Best Use Case	Lightweight automation	Periodic workloads

13. Challenges Encountered

- Pushgateway HTTP 400 conflicts

- Lambda container runtime configuration
- ECS IAM permission issues
- Docker networking configuration
- Prometheus scrape connectivity

All issues were resolved through proper IAM role separation, container runtime alignment, and networking configuration.

---

14. Conclusion

A complete automated monitoring solution was successfully implemented using AWS serverless and container technologies. The system provides scalable, reusable, and observable monitoring with automated alerting capabilities.

The bonus implementation demonstrates container reusability across multiple AWS compute platforms, aligning with modern DevOps best practices.

---

15. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____